

EveryFinance/smart-contracts- Stellar@f342971702b34978b99b9083bba8418954d84cec

Status: COMPLETE
Report Generated: 5/8/2026, 3:59:09 AM
Scan Date: May 8, 2026, 3:49 AM

Findings Summary

Severity	Count
● Critical	1
● High	3
● Medium	9
● Low	4
● Info	2
Total	19

contracts/integration_tests/src/common.rs

● LOW Negative token amounts allow balance manipulation

Description:

`MockToken::transfer` and `MockToken::burn` do not validate `amount >= 0`. Because balances and amounts are `i128`, an authorized caller can pass a negative `amount` to increase their own balance (and/or increase total supply via `burn`). Although this file is under `contracts/integration\_tests` (non-production), this behavior can (a) mask real integration issues because the mock diverges from typical SEP-41 expectations, and (b) would be exploitable if the mock were ever deployed/used outside tests.

Affected Code:

```
pub fn burn(env: Env, from: Address, amount: i128) {
    from.require_auth();
    let b: i128 = env.storage().persistent().get(&TKey::Balance(from.clone())).unwrap_or(0);
    assert!(b >= amount, "burn > balance");
    env.storage().persistent().set(&TKey::Balance(from.clone()), &(b - amount));

    ... (truncated)
```

Recommendation:

Add `assert!(amount >= 0, ...)` (or a shared checked amount validator) to `transfer` and `burn` (and any other amount-taking entrypoints) so the mock matches expected token invariants.

● INFO Anyone can mint unlimited mock tokens

Description:

`MockToken::mint` has no authorization check (and `initialize` can be called/recalled by anyone to overwrite the stored admin, though that admin is not enforced). Any caller can mint arbitrary balances and inflate `TotalSupply`. This is in `contracts/integration\_tests` (non-production), but it weakens test realism and would be a critical issue if the contract were ever deployed/used beyond tests.

Affected Code:

```
pub fn initialize(env: Env, admin: Address) {
    env.storage().instance().set(&TKey::Admin, &admin);
}

pub fn mint(env: Env, to: Address, amount: i128) {
    assert!(amount >= 0, "mint: negative amount");
    let b: i128 = env.storage().persistent().get(&TKey::Balance(to.clone())).unwrap_or(0);

    ... (truncated)
```

Recommendation:

Require an admin auth check in `mint` (and guard `initialize` against re-initialization), e.g., store admin once and enforce `admin.require\_auth()` on privileged operations.

contracts/mock_blend_pool/src/lib.rs

● MEDIUM Deposits can expire and become unwithdrawable

Description:

The contract stores user principal ('Supply(Address)') and 'TotalSupply' in ****persistent**** storage but never bumps their TTL. On Soroban, persistent entries can expire if not extended; once 'Supply(from)' expires, 'withdraw' reads it as '0' and permanently prevents the user from withdrawing their deposited tokens, while the pool still holds the tokens. Exploit path: a user supplies tokens and then remains inactive beyond the network's persistent-entry TTL; the 'Supply(from)' entry expires; any later withdraw attempt fails with 'InsufficientSupply', effectively locking funds in the pool forever (no recovery path exists in this contract).

Affected Code:

```
env.storage()
  .persistent()
  .set(&DataKey::Supply(from.clone()), &(bal + req.amount));
...
env.storage()
  .persistent()
  .set(&DataKey::TotalSupply, &(total + req.amount));
...
env.storage()
  .persistent()
  .get(&DataKey::Supply(account))
  .unwrap_or(0)
```

Recommendation:

Bump TTL for 'Supply(from)' and 'TotalSupply' on write (and optionally on read) using Soroban storage TTL bump helpers (e.g., bump for a configured number of ledgers) so balances cannot silently expire while funds remain escrowed.

● LOW Unbounded per-address state growth enables bloat

Description:

Every unique `from` that supplies creates a new persistent `Supply(Address)` entry. There is no minimum deposit, no cap on number of suppliers, and state only shrinks if users fully withdraw. An attacker can create many addresses and supply a dust amount from each, growing persistent state without bound across transactions. Exploit path: attacker funds many accounts, approves the pool, and calls `submit` with `REQUEST\_SUPPLY` for each account (leaving nonzero balances). This increases ledger state size and can raise ongoing footprint/fee costs for interacting with the contract, potentially degrading availability/economics for legitimate users.

Affected Code:

```
enum DataKey {
    Admin,
    Token,
    Supply(Address),
    TotalSupply,
}
...
env.storage()
    .persistent()
    .set(&DataKey::Supply(from.clone()), &(bal + req.amount));
```

Recommendation:

Consider adding a minimum deposit threshold, supplier caps, or a mechanism to consolidate/prune dust positions; alternatively use temporary storage with explicit renewal rules if long-lived balances are not required (or document that this is test-only/mock behavior).

contracts/strategies/phoenix_lp/src/lib.rs

● HIGH Public strategy initialization can be front-run and permanently squatted

Description:

Multiple strategy contracts expose a publicly callable `initialize` that accepts an arbitrary `vault` address and derives authorization from `vault.get\_manager()`. Because the vault address is user-supplied, an attacker can deploy or choose a contract that returns an attacker-controlled manager, initialize the strategy first, and permanently lock the instance to attacker-chosen vault/asset/router/pool values via `AlreadyInitialized`. This bricks the intended strategy deployment and prevents the real vault/manager from initializing or using the strategy correctly.

Affected Code:

```
pub fn initialize(
    env: Env,
    vault: Address,
    asset_a: Address,
    asset_b: Address,
    phoenix_pool: Address,
    manager: Address,
    name: String,
) {
    if is_initialized(&env) {
        panic_with_error!(&env, PhoenixLpError::AlreadyInitialized);
    }

    let vault_manager: Add
    ... (truncated)
```

Recommendation:

Make initialization atomic and bind it to a trusted, unspoofable authority. Deploy and initialize in one transaction, or restrict `initialize` to a factory/deployer/constructor-bound address. Alternatively, require a verifiable vault identity (e.g. immutable expected vault, factory registry, or vaul... (truncated))

● MEDIUM Unused deposited tokens can become permanently stranded

Description:

`deposit\_liquidity` pulls `amount\_a`/`amount\_b` from the vault into the strategy, then calls `provide\_liquidity`. Many AMMs consume tokens according to pool ratios and may not use the full desired amounts; any unused tokens are typically returned to the depositor (here: the strategy contract address). This strategy never refunds or otherwise accounts for any leftover `asset\_a`/`asset\_b` balances remaining in the strategy after `provide\_liquidity`. The strategy's internal accounting (`total\_shares`) tracks only minted share tokens and `get\_value` values the position only via share tokens (and optionally pool reserves), so any stranded balances will not be reflected in NAV. This can lead to: - Value under-reporting and incorrect vault share pricing (users can be diluted), and/or - Permanent loss of operational control over those tokens unless another "sweep"/"rescue" mechanism exists elsewhere. An attacker-controlled or simply ratio-shifted pool can intentionally leave dust/leftovers on repeated deposits to accumulate stranded balances over time.

Affected Code:

```
// Pull tokens from vault.
token::Client::new(&env, &asset_a).transfer(&from, &strategy, &amount_a);
token::Client::new(&env, &asset_b).transfer(&from, &strategy, &amount_b);

// Approve to pool.
token::Client::new(&env, &asset_a).approve(&strategy, &pool, &amount_a, &expiry);
token::Client::new(&env
... (truncated)
```

Recommendation:

After `provide\_liquidity`, measure the strategy's `asset\_a`/`asset\_b` balances and either:

- Transfer any residual balances back to the vault, or
- Include idle balances in `get\_value` and expose a manager-only `sweep` to return them to the vault.

Also consider recording actual amounts consumed if ... (truncated)

contracts/trade_guards/soroswap/src/lib.rs

● MEDIUM Exact-out swaps lack any slippage enforcement

Description:

The contract documentation and module-level comments state that exact-out swaps must be bounded by a MAX_SLIPPAGE_BPS headroom check, but `validate_swap_exact_out` only validates caller=vault, positive amounts, path length, and whitelist membership. It never obtains an on-chain quote (or accepts a quoted value) and never checks that `max_in` is within the permitted headroom relative to a trusted quote. Exploitation path: if the vault (or a strategy/integration feeding swap params to the vault) uses `swap_tokens_for_exact_tokens` flows, an attacker can influence `max_in` to be set extremely high, then manipulate the DEX price so the router executes at a very poor rate but still within the inflated `max_in`. Because this guard approves the call, the vault can overpay and lose funds up to `max_in`. This is particularly high-signal because the repo's docs explicitly claim exact-out slippage enforcement exists (docs/contracts/soroswap_trade_guard.md), suggesting integrators may rely on this protection when it is not actually present.

Affected Code:

```
pub fn validate_swap_exact_out(
    env: Env,
    caller: Address,
    amount_out: i128,
    max_in: i128,
    path: Vec<Address>,
) {
    env.storage()
        .instance()
        .extend_ttl(INSTANCE_LIFETIME_THRESHOLD, INSTANCE_BUMP_AMOUNT);

    caller.require_auth();
    if caller != get_vault(
        ... (truncated)
```

Recommendation:

Implement the exact-out headroom/slippage check as documented, using a trusted on-chain quote from the stored `strategy` (e.g., `quote_exact_out(amount_out, path)` returning `quoted_in`), and enforce `(max_in - quoted_in) * 10_000 <= quoted_in * MAX_SLIPPAGE_BPS` with checked math. Alternatively, re... (truncated)

● LOW Initialization can be front-run to permanently lock guard

Description:

`initialize` is not a constructor and can be called by any transaction that can satisfy its auth requirements. While it requires auth from the vault's on-chain manager (`vault.get\_manager()`) and from the provided `manager` parameter, it does not require authorization from the `vault` address itself, nor does it bind `manager` to the vault's manager. If a guard contract is deployed at a predictable address and initialization is not performed atomically in the same transaction as deployment, an attacker can front-run by calling `initialize` first with a vault they control (and corresponding auths). This sets the stored vault/manager/strategy/whitelist and permanently prevents legitimate initialization (`AlreadyInitialized`), effectively bricking the guard instance for its intended vault. Impact depends on deployment workflow (whether guard contract IDs are predictable and whether init is atomic).

Affected Code:

```
pub fn initialize(
    env: Env,
    vault: Address,
    manager: Address,
    tokens: Vec<Address>,
    strategy: Address,
) {
    if is_initialized(&env) {
        panic_with_error!(&env, SoroswapGuardError::AlreadyInitialized);
    }

    let vault_manager: Address = env.invoke_contract(

... (truncated)
```

Recommendation:

Prefer using a `\_\_constructor` (called atomically during `CreateContract`) or require `vault.require\_auth()` so only the vault contract itself can initialize the guard. Additionally, consider enforcing `manager == vault\_manager` (or otherwise binding manager selection to vault governance) to reduce ... (truncated)

contracts/trade_guards/phoenix/src/lib.rs

● MEDIUM Slippage cap bypassed via spoofed quoted_out

Description:

`validate\_swap` and `validate\_swap\_exact\_in` enforce the max slippage limit using the caller-provided `quoted\_out` value rather than a quote fetched from a trusted on-chain source (e.g., the strategy/router). If the vault (or an upstream caller) forwards a user-controlled `quoted\_out`, an attacker can bypass the slippage cap by setting `quoted\_out == min\_out` (diff=0), while choosing an arbitrarily low `min\_out`. The guard would then approve a trade that can execute with effectively unlimited slippage, potentially causing vault value loss. Exploit path (when `quoted\_out` is caller-influenced): a trader requests a swap with very low `min\_out` and supplies `quoted\_out = min\_out`; the guard's check passes, the vault forwards the low `min\_out` to the DEX/router, and the trade executes at an extremely unfavorable rate without being blocked by the guard's intended slippage policy.

Affected Code:

```
pub fn validate_swap(
    env: Env,
    caller: Address,
    amount_in: i128,
    min_out: i128,
    quoted_out: i128,
    operations: Vec<SwapOperation>,
) {
    ...
    Self::check_slippage(min_out, quoted_out, &env);
}

pub fn validate_swap_exact_in(
    env: Env,
    caller: Address,
    amount_
    ... (truncated)
```

Recommendation:

Do not accept `quoted\_out` as an external parameter for slippage enforcement. Instead, have the guard fetch `quoted\_out` internally from a trusted contract (strategy/router wrapper) using `env.invoke\_contract`, as done in the Soroswap guard, and/or bind `quoted\_out` to a trusted oracle price. If `qu... (truncated)

● LOW Vault cannot call Phoenix validate_swap_exact_in

Description:

The vault's `guard_validate` helper invokes a guard's `validate_swap_exact_in(vault, amount_in, min_out, path)` with 4 arguments. This Phoenix guard's `validate_swap_exact_in` entrypoint requires an additional `quoted_out` argument. If this guard is configured for a strategy in the vault, the cross-contract call will fail due to argument mismatch, preventing spot trades (denial of service for trading through that strategy). Exploit path: if a privileged actor (manager) configures this Phoenix guard as the strategy's trade guard, any subsequent `vault.execute_trade` will attempt to invoke `validate_swap_exact_in` with the vault's 4-arg convention and trap/fail, blocking trades.

Affected Code:

```
pub fn validate_swap_exact_in(
  env: Env,
  caller: Address,
  amount_in: i128,
  min_out: i128,
  path: Vec<Address>,
  quoted_out: i128,
) { ... }
```

Recommendation:

Align the Phoenix guard interface with the vault's guard convention: change `validate_swap_exact_in` to accept only `(caller, amount_in, min_out, path)` and fetch `quoted_out` internally from the trusted strategy/router, or update the vault's `guard_validate` wrapper to pass the required `quoted_out...` (truncated)

contracts/vault/src/lib.rs

● MEDIUM Residual token allowance after strategy deposit

Description:

The vault grants the strategy a token allowance via `token::Client::approve(vault, strategy, amount, expiry)` and then calls `strategy.deposit(...)` / `strategy.deposit_liquidity(...)`, but it never revokes or zeroes the allowance afterward. If the strategy contract (or an account able to act as the strategy contract, e.g., via an upgrade if the strategy is upgradeable) pulls less than the approved amount during the deposit (common for LP adds due to pool ratio/slippage, or due to strategy-side rounding), the leftover allowance remains valid until `expiry` (ledger+100). During that window, the strategy can later call `transfer_from(vault, strategy, leftover)` (or equivalent) to drain additional vault funds without any further vault authorization. This is particularly relevant for `invest_lp`, where exact amounts are approved but the LP router may only consume a subset. Attack path: 1) Manager (or compromised manager) invests into a strategy that does not fully consume the approved amounts (or a strategy that intentionally under-consumes). 2) The deposit succeeds, leaving non-zero remaining allowance on one or both tokens. 3) Before allowance expiry, the strategy pulls the remaining allowance from the vault, stealing assets. Even if strategies are “whitelisted”, this pattern increases blast radius from strategy bugs/misconfiguration and is a common footgun in cross-contract token flows.

Affected Code:

```
// invest(): approve then call strategy_deposit
let expiry = env.ledger().sequence() + 100;
token::Client::new(&env, &base_asset).approve(&vault, &strategy, &amount, &expiry);
let units = strategy_deposit(&env, &strategy, amount, &vault);

// invest_lp(): approve two tokens then call strategy_deposi
... (truncated)
```

Recommendation:

After the strategy call returns, explicitly clear any remaining allowance by calling `approve(vault, strategy, 0, expiry)` (or set allowance to 0 with a fresh expiry) for each token approved. For LP invests, consider approving exact amounts, then revoking, and/or verifying post-call vault balances d... (truncated)

● MEDIUM Auto-unwind may withdraw wrong strategy units

Description:

During `withdraw`, if the vault lacks enough base-asset balance, it auto-unwinds single-asset strategies by calling `strategy\_withdraw(..., pull, vault, vault)` where `pull` is computed from `strategy\_value\_in\_nav` (NAV-denominated value). However `strategy\_value\_in\_nav` may apply oracle conversion (`raw\_v \* price / PRICE\_PRECISION`) for strategies with `StrategyPriceToken` configured. In that case, the NAV-denominated value is not necessarily the same unit that the strategy's `withdraw(amount, from, to)` expects (it may expect raw position units or underlying-token units depending on strategy implementation). If a strategy's `get\_value()` is not already expressed in base-asset units, the vault can send an incorrect `amount` to `withdraw`, causing under-withdraw (DoS/insufficient liquidity revert) or over-withdraw (if the strategy interprets `amount` differently), breaking accounting and potentially enabling a malicious/buggy strategy to return fewer base tokens than the vault assumes. Attack path (realistic): 1) Manager configures an oracle + strategy price token for a strategy whose `withdraw` parameter is not base-asset-denominated (or whose `get\_value` returns non-base units). 2) Users attempt to withdraw; vault enters auto-unwind path. 3) Vault calls `strategy.withdraw(pull\_in\_nav\_units, ...)` which the strategy interprets in a different unit, returning insufficient base asset. 4) Withdraw reverts with `InsufficientLiquidity`, effectively freezing withdrawals until manager manually unwinds correctly. This is primarily a safety/availability risk but can also cause value loss if the mismatch leads to over-redemption or incorrect share price assumptions.

Affected Code:

```
// withdraw(): computes per-strategy sv in NAV units and passes as amount to strategy_withdraw
let sv = Self::strategy_value_in_nav(&env, &vault, &s);
// ...
let portion = shortfall.saturating_mul(sv) / total_sa_value;
let pull = if portion > sv { sv } else { portion };
if pull > 0 {
    strategy_wi
    ... (truncated)
```

Recommendation:

Enforce a strict unit invariant: require that any strategy eligible for auto-unwind returns `get\_value` in base-asset units and that `withdraw(amount)` expects base-asset units. If oracle conversion is needed for NAV, then auto-unwind should instead use a strategy-specific redemption-by-value interf... (truncated)

contracts/integration_tests/src/test_vault_soroswap_lp.rs

● INFO Auth-mocking hides missing require_auth coverage

Description:

The test harness enables `env.mock_all_auths_allowing_non_root_auth()`, which makes authorization succeed implicitly for non-root callers. This can cause integration tests to pass even if production contract endpoints are missing or mis-scoped `require_auth` checks (e.g., manager-only functions like pause/unpause, strategy initialization, vault strategy setting). As a result, an actual authorization bypass could ship undetected because the tests never exercise real auth requirements.

Affected Code:

```
let env = Env::default();
env.mock_all_auths_allowing_non_root_auth();
```

Recommendation:

For tests intended to validate access control, avoid global auth-mocking. Prefer explicit `mock_auth` / recorded auth expectations per call (or run a subset of tests without auth-mocking) so missing `require_auth` triggers failures.

contracts/oracle/src/lib.rs

● HIGH Re-callable constructor lets attacker seize admin

Description:

`\_\_constructor` is a public entrypoint and can be invoked after deployment, but it does not check whether the contract is already initialized (despite `OracleError::AlreadyInitialized` existing) and does not require any authorization. Any user can call `\_\_constructor(env, attacker\_addr)` to overwrite the stored admin, then call `set\_price` / `set\_max\_age\_ledgers` as the new admin to manipulate oracle prices (and any downstream vault logic that trusts this oracle).

Affected Code:

```
pub fn __constructor(env: Env, admin: Address) {
    set_admin(&env, &admin);
    set_max_age_ledgers(&env, DEFAULT_MAX_AGE_LEDGERS);
    env.storage()
        .instance()
        .extend_ttl(INSTANCE_LIFETIME_THRESHOLD, INSTANCE_BUMP_AMOUNT);
}
```

Recommendation:

Make initialization one-time: in `\_\_constructor`, check whether `DataKey::Admin` is already set and `panic\_with\_error!(..., OracleError::AlreadyInitialized)` if so (or store an explicit init flag). Alternatively, gate any re-initialization behind `get\_admin().require\_auth()` and only allow it if alr... (truncated)

contracts/factory/src/lib.rs

● CRITICAL Public constructor allows contract reinitialization takeover

Description:

The contract exposes `pub fn __constructor(...)` without any “already initialized” guard. In Soroban, this function remains callable after deployment like any other entrypoint. Because it only checks `admin.require_auth()` against the *parameter* `admin`, any attacker can call `__constructor(env, attacker_addr)`, satisfy `require_auth()` for their own address, and overwrite the stored admin and reset `VaultCount` to 0. Impact: attacker becomes admin and can subsequently register/remove vaults and change admin arbitrarily; additionally, resetting `VaultCount` can desynchronize instance vs persistent state and potentially break registry reads/removals.

Affected Code:

```
pub fn __constructor(env: Env, admin: Address) {
    admin.require_auth();
    env.storage()
        .instance()
        .extend_ttl(INSTANCE_LIFETIME_THRESHOLD, INSTANCE_BUMP_AMOUNT);

    set_admin(&env, &admin);
    set_vault_count(&env, 0);
}
```

Recommendation:

Add an initialization guard in instance storage (e.g., an `Initialized` boolean/DataKey) and `panic` if `__constructor` is called when already initialized. Alternatively, only allow `__constructor` when `Admin` is absent and avoid resetting existing state. Keep `require_auth` but do not rely on an a... (truncated)

contracts/factory/src/storage.rs

● MEDIUM Registry entries can expire and corrupt index

Description:

Persistent registry entries (`VaultByIndex`, `VaultPosition`, `IsRegistered`) rely on per-key `extend\_ttl` only when that *specific* key is accessed. Any vault entry not read/written for ~30 days (per `PERSISTENT\_BUMP\_AMOUNT` / `PERSISTENT\_LIFETIME\_THRESHOLD`) can expire and be deleted by the network. Impact paths once some entries expire: - `get\_vault\_by\_index` / `get\_vault\_position` will `unwrap()` a missing value and trap, breaking `get\_vaults`, `remove\_vault`, and any other logic that assumes the index is intact. - `get\_is\_registered` returns `false` when the membership key is missing (including due to TTL expiry). This can make an already-indexed vault appear “not registered”, allowing re-registration (admin-only, but realistic operationally) and creating duplicates/inconsistent state. This is an availability/state-integrity issue that can brick the registry over time unless an off-chain keeper continuously pages through and touches all entries often enough.

Affected Code:

```
pub const PERSISTENT_BUMP_AMOUNT: u32 = 518_400; // ≈ 30 days
pub const PERSISTENT_LIFETIME_THRESHOLD: u32 = 259_200; // ≈ 15 days

fn bump_persistent(env: &Env, key: &DataKey) {
    env.storage()
        .persistent()
        .extend_ttl(key, PERSISTENT_LIFETIME_THRESHOLD, PERSISTENT_BUMP_AMOUNT);

    ... (truncated)
```

Recommendation:

Prevent silent expiry of registry keys. Common mitigations:

- Provide/require a maintenance endpoint that bumps TTLs for a configurable range of indices (pagination) and have ops/keepers call it regularly.
- Alternatively, increase persistent TTL parameters to a horizon that matches intended perma... (truncated)

contracts/share_token/src/lib.rs

● HIGH Self transfer_from inflates balance without supply

Description:

`transfer_from` does not handle the `from == to` case. When `to` equals `from`, the function calculates `new_from_balance = from_balance - amount` and `new_to_balance = to_balance + amount` using the same original balance, then writes the balance twice to the same storage key. The second write overwrites the first, leaving the account with `from_balance + amount` (i.e., balance increases by `amount`) while `total_supply` is unchanged. Exploit path: an attacker with any non-zero share balance can (1) `approve(attacker, attacker, amount=balance, ...)` then (2) call `transfer_from(spender=attacker, from=attacker, to=attacker, amount=balance)` to double their balance. Repeating allows unbounded balance inflation (counterfeit shares) without admin privileges, enabling fraud/griefing and breaking invariants for any integrator relying on balances; additionally, these “phantom” shares can later cause burns/redemptions to revert due to `total_supply` underflow.

Affected Code:

```
pub fn transfer_from(env: Env, spender: Address, from: Address, to: Address, amount: i128) {  
    ...  
    let from_balance = get_balance(&env, &from);  
    if from_balance < amount {  
        panic_with_error!(&env, ShareTokenError::InsufficientBalance);  
    }  
  
    let new_allowance = allowance  
  
    ... (truncated)
```

Recommendation:

In `transfer_from`, add an early `if from == to { ... }` branch that only decrements allowance (if applicable) and emits the transfer event, without modifying balances. Alternatively, ensure only one balance write occurs for `from == to` and that it preserves the original balance. Add a unit test fo... (truncated)

contracts/strategies/blend/src/lib.rs

● MEDIUM Withdraw returns requested amount, not actual

Description:

`withdraw` always returns the input `amount` even though the docstring states the actual withdrawn amount may differ due to Blend rounding. If the Blend pool rounds down (or performs partial fulfillment), the vault or off-chain integrators that trust this return value can over-credit withdrawals or mis-account TVL, potentially enabling share-price/accounting manipulation (especially if other parts of the system use returned values rather than observed token balances). Exploit path: a manager (or any component relying on this return) triggers `withdraw` for an amount that Blend fulfills with a smaller transfer; the strategy reports the higher amount, causing the caller to believe funds were received that were not. If this value is used in downstream accounting (fees, NAV updates, events consumed by off-chain accounting), it can lead to incorrect state or payouts.

Affected Code:

```
pub fn withdraw(env: Env, amount: i128, from: Address, to: Address) -> i128 {
    ...
    blend_submit(
        &env,
        &protocol,
        &strategy_addr,
        &strategy_addr,
        &to,
        requests,
    );

    amount
}
```

Recommendation:

Return the actual amount transferred/withdrawn from Blend (if `submit` returns it, decode and return it), or verify the recipient's token balance change and return that. Alternatively document that `withdraw` returns the requested amount and ensure all callers use observed balances/NAV queries inste... (truncated)

contracts/strategies/soroswap_lp/src/lib.rs

● MEDIUM LP valuation can be manipulated via oracle output

Description:

`get\_value` trusts an external oracle contract to return `i128` prices and performs reserve-decomposition valuation using those values without validating basic invariants (e.g., price > 0) and with `saturating\_\*` arithmetic. If the configured oracle can be influenced by an attacker (e.g., is upgradable, permissionless, or reads manipulable on-chain state), an attacker can cause extreme/negative prices that distort the strategy's reported value: - Negative (or artificially low) prices reduce `get\_value`, potentially reducing vault NAV and letting an attacker mint disproportionately many vault shares on deposit, then redeem later after prices normalize (classic share-price manipulation). - Extremely high prices can saturate to `i128::MAX` due to `saturating\_mul`, producing non-linear/capped valuations that may break vault invariants and downstream guards. Whether this is exploitable depends on the oracle's trust model and whether the vault's value-manipulation/NAV guards are always enabled; the strategy itself provides no safety rails.

Affected Code:

```
let oracle_client = OracleAdapter::new(&env, &oracle);
let price_a = oracle_client.get_price(&asset_a);
let price_b = oracle_client.get_price(&asset_b);

let pool_value_a = reserve_a.saturating_mul(price_a) / PRICE_PRECISION;
let pool_value_b = reserve_b.saturating_mul(price_b) / PRICE_PRECISION;
le
... (truncated)
```

Recommendation:

Defensively validate oracle outputs and arithmetic:

- Require `price\_a > 0 && price\_b > 0` (and optionally upper bounds) and revert otherwise.
- Consider using checked math and reverting on overflow rather than saturating, so valuations fail closed.
- If feasible, use a manipulation-resistant oracle... (truncated)